

Chapter 04

Programming Principles

Object-Oriented Programming

Classes and Objects

Mr. SOK Pongsametrey
metreysk@gmail.com

This week we explore the foundations of Object-Oriented Programming (OOP), showing how classes and objects directly support Programming Principles such as modularity, separation of concerns, reusability, maintainability, and DRY.

ABOUT

What is Object-Oriented Programming?

- A programming paradigm based on **objects**
- Models real-world entities using:
 - **Attributes** (fields/properties)
 - **Behaviors** (methods/functions)
- Core benefits: modularity, reusability, maintainability
- **Principle tie-in: OOP naturally supports separation of concerns and DRY principles**

Core OOP Concepts

(Today's Focus)

- **Class** – Blueprint or template for creating objects
- **Object** – An instance of a class (actual data in memory)
- **Fields** – Variables that store object data (state)
- **Methods** – Functions that define object behavior (actions)
- **Constructor** – Special method to initialize objects
- **Encapsulation** – Controlling access to object data

What is a Class?

- A blueprint or template for creating objects
- Defines what properties (fields) an object will have
- Defines what behaviors (methods) an object can perform
- **Real-world analogy:** Class = House blueprint, Objects = Actual houses built from that blueprint
- **Principle tie-in: Classes promote modularity and reusability**

Defining a Class in Java

```
public class Student {  
    // Fields (attributes)  
    String name;  
    int age;  
    String major;  
  
    // Method (behavior)  
    void study() {  
        System.out.println(name + " is studying " + major);  
    }  
  
    void displayInfo() {  
        System.out.println("Name: " + name + ", Age: " + age);  
    }  
}
```

- **Principle tie-in: Bundles related data and actions together (separation of concerns)**

Creating and Using Objects

```
// Creating objects (instantiation)
Student student1 = new Student();
Student student2 = new Student();

// Setting field values
student1.name = "Sopheha";
student1.age = 22;
student1.major = "Computer Science";

// Calling methods
student1.study();
student1.displayInfo();
```

- **Principle tie-in: Reuse the same class blueprint for multiple instances (DRY)**

The `this` Keyword

```
public class Student {  
    String name;  
    int age;  
  
    void setName(String name) {  
        this.name = name; // 'this' refers to current  
object  
    }  
  
    void introduce() {  
        System.out.println("Hi, I'm " + this.name);  
    }  
}
```

- this refers to the current object
- Resolves naming conflicts between parameters and fields
- **Principle tie-in: Improves code clarity and prevents ambiguity**

Constructors in Java

```
public class Student {  
    String name;  
    int age;  
    String major;  
  
    // Default constructor  
    public Student() {  
        name = "Unknown";  
        age = 0;  
        major = "Undeclared";  
    }  
  
    // Parameterized constructor  
    public Student(String name, int age, String major) {  
        this.name = name;  
        this.age = age;  
        this.major = major;  
    }  
}
```

- **Principle tie-in: Constructors ensure proper initialization (reliability) and reduce repetitive setup code (DRY)**

Using Constructors

```
// Using default constructor
Student student1 = new Student();

// Using parameterized constructor
Student student2 = new Student("Dara", 23, "IT");
Student student3 = new Student("Pisey", 21, "Business");

// No need to set fields manually!
student2.study(); // "Dara is studying IT"
```

- **Principle tie-in: Cleaner, more maintainable object creation**

Encapsulation and Access Modifiers

```
public class BankAccount {  
    private double balance; // private = hidden from  
    outside  
    private String accountNumber;  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
  
    public double getBalance() {  
        return balance; // controlled access  
    }  
}
```

- **Access modifiers:** private, public, protected
- **Principle tie-in:** Encapsulation improves security, maintainability, and separation of concerns

Real-world AI Example - ChatMessage Class

```
public class ChatMessage {  
    private String sender;  
    private String content;  
    private long timestamp;  
  
    public ChatMessage(String sender, String content) {  
        this.sender = sender;  
        this.content = content;  
        this.timestamp = System.currentTimeMillis();  
    }  
  
    public String formatMessage() {  
        return "[" + sender + "]: " + content;  
    }  
  
    public boolean isFromUser() {  
        return !sender.equals("AI");  
    }  
}
```

- **Principle tie-in: Modularity allows easy feature expansion (e.g., adding sentiment analysis, translation)**

Multiple Objects in Action

```
public static void main(String[] args) {  
    // Create multiple students  
    Student[] students = {  
        new Student("Alice", 20, "CS"),  
        new Student("Bob", 22, "IT"),  
        new Student("Carol", 21, "Business")  
    };  
  
    // Each object maintains its own state  
    for (Student s : students) {  
        s.study();  
        s.displayInfo();  
    }  
}
```

- **Principle tie-in: Objects encapsulate their own data independently**

Lab Activity

Create a Product Class

- Private fields: name, price, quantity
- Constructor to initialize all fields
- Public methods: displayInfo(), updatePrice(), isInStock()
- Getter methods for accessing private fields
- Create 3 product objects and test all methods
- **Focus: Apply encapsulation and observe how it improves data protection**

Exercise

Create a Book Class

- Private fields: title, author, pages, isRead
- Constructor and appropriate getter/setter methods
- Methods: markAsRead(), getReadingStatus(), displayDetails()
- Create a small library of 5 books in main()

Reflection Questions:

- How does encapsulation improve your class design?
- Which Programming Principles did you apply?

Due: Next week

Wrap-up and Q&A

- Classes are blueprints; objects are instances
- Constructors ensure proper initialization
- Encapsulation protects data and controls access
- OOP directly supports Programming Principles: DRY, modularity, separation of concerns, maintainability
- **Next Week: OOP Part 2 - Inheritance and Polymorphism**